

Contents

SmolLM2 Zybo 5일 DMA/Agent 선행 실행 가이드 v5	2
0A. 문서 원칙, 사람용/Agent용 분리	2
0. 지금 프로젝트 판정	2
1. 최종 하드웨어/소프트웨어 흐름	4
2. Codex/AI agent 권한과 안전 규칙	4
3. 선행 실행표	5
S00. 낡은 AXI-Lite 데이터 경로 격리/삭제	5
육하원칙	5
Codex 프롬프트 - 삭제/격리	6
검증 프롬프트	6
S01. DMA 기반 하드웨어 구조 작성	7
육하원칙	7
Codex 프롬프트 - DMA 구조 및 라우팅 복구	7
검증 프롬프트	10
S01.5. GPTalk.xpr 단일 active project 통합	10
육하원칙	10
Codex 프롬프트 - GPTalk.xpr 통합	11
검증 프롬프트	11
S02. bitstream/XSA 생성과 routing/timing	12
육하원칙	12
Codex 프롬프트	12
검증 프롬프트	12
S03. PetaLinux/SD/serial 자동화	13
육하원칙	13
Codex 프롬프트	13
검증 프롬프트	14
S03.5. 부트 복구 baseline 판정과 FSBL 계보 고정	14
목적	14
Codex 프롬프트 - boot baseline 고정	14
S04. 보드 UIO/register/BRAM/DMA-register smoke	15
목적	15
Codex 프롬프트	15
v5 현재 S04 결과	16
검증 프롬프트	16
S04.5. DMA buffer provider 확보	16
목적	16
작업 분리 원칙	16
S04.5A Codex 프롬프트 - 보드-only 임시 mem= 테스트	17
S04.5C Codex 프롬프트 - 보드-only /dev/mem carveout smoke	17
S04.5B Codex 프롬프트 - SD-only 영구 bootargs 반영	18
S04.5D fallback	19
S04.5 검증 프롬프트	19
S05. C gemv_hw_test로 fake_gemv FPGA PASS	19
진입 조건	19
Codex 프롬프트	19
검증 프롬프트	20
S06. Runtime 모든 GEMV FPGA backend화	20
Codex 프롬프트	20
검증 프롬프트	21
S07. HDMI console 최종 demo	21
Codex 프롬프트	21
검증 프롬프트	22

S08. Freeze와 발표 리허설	22
4. 팀원에게 넘길 것	22
5. 최종 판정표	23
6. 절대 금지 문구	23
7. v5 반영 이력	23

SmolLM2 Zybo 5일 DMA/Agent 선형 실행 가이드 v5

목적: v4 전체 흐름을 유지하면서, 실제 진행 중 추가된 S01.5 프로젝트 통합, S03 부트 복구 baseline, S04 통과 결과, S04.5 DMA buffer provider 단계를 반영한 **전체 선형 실행 가이드**다.

대상: 사용자, 팀원 A(runtime/Linux), 팀원 B(Vivado/PetaLinux), Codex/AI agent.

최종 목표: Zybo Z7-20 Linux console에서 `smolllm2_chat --backend fpga --require-fpga`가 실행되고, Transformer 내부 Q8_0 GEMV가 전부 FPGA backend를 탄다.

v5 핵심: 이 문서는 v4의 대체 축약본이 아니라, v4 전체 가이드에 실제 진행 결과와 x.5 보강 단계를 병합한 전체판이다.

0A. 문서 원칙, 사람용/Agent용 분리

문서 난립을 막기 위해 역할을 고정한다.

사람용 문서:

README.md
 프로젝트 입구. 목표, active Vivado project, 최종 데모 명령만 짧게.
 docs/00_ACTIVE_KR.md
 현재 상태판. 지금 active project, 현재 단계, 다음 명령 1~3개만.
 docs/VIVADO_GUI_KR.md
 사람이 Vivado GUI에서 무엇을 눌러야 하는지 설명.

Codex/agent용 문서:

prompts/*.md
 docs/internal/*.md
 이 v5 가이드의 Codex 프롬프트 블록

로그/리포트:

logs/*.log
 reports/*.rpt

금지:

- README에 긴 Tcl 본문을 넣지 않는다.
- README에 Codex 프롬프트 전문을 넣지 않는다.
- README에 빌드 로그를 넣지 않는다.
- docs/00_ACTIVE_KR.md를 장문 설계 문서로 만들지 않는다.
- 사람에게 보여줄 Vivado 작업은 Tcl-first가 아니라 GUI-first로 설명한다.
- 새 문서를 만들기 전에 README / ACTIVE / GUI / prompts / logs 중 어느 성격인지 판단한다.

이 문서는 **전체 실행 흐름과 Codex 프롬프트를 보존하는 작업 가이드**다. 사람이 매번 처음부터 읽는 README가 아니다.

0. 지금 프로젝트 판정

현재 업로드 프로젝트 기준 판정은 다음이다.

살릴 것:

vivado_ip/rtl/gemv_q8_0_stream_core.v
 vivado_ip/tb/tb_gemv_q8_0_stream_core.sv
 scripts/run_gemv_sim.tcl
 golden/fake_gemv/*
 fpga_layout/q8_0_lane16/*
 runtime_c/*

버릴 것 또는 active build에서 제거할 것:

AXI-Lite STREAM_DATA 반복 write로 weight/scale을 보내는 구조

AXI-Lite INPUT_DATA 반복 write로 input vector를 보내는 구조
AXI-Lite RESULT_DATA 반복 read로 output vector를 읽는 구조
DMA 없는 create_zybo_gemv_hw.tcl 최종 경로
smoke-only bitstream을 GEMV 성공물로 취급하는 것

현재 bitstream:

version/status register smoke용이다.
full GEMV datapath와 DMA path가 없다.

현재 최종 구조 적합성:

부적합. DMA 기반으로 갈아엎어야 한다.

0.1 v5 현재 확정 상태

현재 진행 결과는 다음처럼 반영한다.

S03 boot recovery baseline:

PASS

기준 BOOT:

artifacts/boot_tests/test_c_s03_fsbl_active_bit_s03_uboot/BOOT.BIN

구성:

known-good S03 FSBL

active GPTalk_dma.bit

S03 U-Boot

결과:

DONE LED on

Linux root prompt 도달

/sys/class/uio 노드 노출

/opt/smollm2_zybo 존재

폐기:

custom/recovery FSBL

결론:

PetaLinux full rebuild는 S03 boot recovery에는 불필요

S04 UIO/register/BRAM/DMA-register smoke:

PASS

axi_dma: uio0 0x40400000 size 0x10000

input_bram: uio1 0x42000000 size 0x10000

gemv_ctrl: uio5 0x43ca0000 size 0x1000

gemv_ctrl VERSION=0x000a0001 STATUS=0 ERROR=0

input_bram write/readback PASS

AXI DMA reset/status register PASS

bus error/kernel oops 없음

현재 BLOCKED:

S05 fake_gemv DMA transfer

차단 이유:

/dev/udmabuf* 없음

/dev/dma_proxy* 없음

/proc/device-tree/reserved-memory 없음

/sys/class/dma_heap 없음

CMA는 있지만 user-space DMA physical buffer provider로 노출되지 않음

다음 단계:

S04.5 DMA buffer provider 확보

현재부터는 부팅/PetaLinux가 아니라 DMA buffer provider → fake_gemv → SmolLM2 runtime 순서다.

프로젝트 목표는 낮추지 않는다.

금지:

q_proj 하나만 FPGA로 보내고 성공 처리

fake_gemv 전용 하드코딩 IP

scale 적용 제거

mode=0/mode=1 제거
16-lane을 임의로 박살내기
CPU fallback을 최종 성공으로 처리
AXI-Lite로 weight 전체를 반복 전송

1. 최종 하드웨어/소프트웨어 흐름

PC/Vivado/Codex

- > GEMV stream core 유지
- > AXI DMA + AXIS FIFO + GEMV DMA top 구성
- > bitstream 생성
- > XSA export

PetaLinux/boot packaging/Codex

- > XSA 반영
- > device tree/DMA/UIO 또는 /dev/mem 접근 준비
- > BOOT.BIN/image.ub/SD 구성

Zybo board/Codex + 사용자

- > 사용자는 SD 삽입/전원/부팅만 수행
- > Codex는 serial terminal로 보드 조작
- > C gemv_hw_test 실행
- > fake_gemv FPGA PASS

Runtime/Codex

- > SmolLM2-135M Q8_0 GGUF load
- > 모든 Q8_0 GEMV call을 gemv_backend_run()으로 통과
- > backend=fpga, require-fpga 모드
- > HDMI + USB keyboard console에서 demo

최종 demo 명령:

```
./smolllm2_chat \  
--model /mnt/sd/SmolLM2-135M-Instruct-Q8_0.gguf \  
--backend fpga \  
--require-fpga \  
--max-new-tokens 16
```

최종 출력에 반드시 있어야 하는 숫자:

```
total_gemv_calls: N  
fpga_gemv_calls: N  
cpu_gemv_fallbacks: 0
```

2. Codex/AI agent 권한과 안전 규칙

이번 프로젝트는 AI agent에 적극 위임한다. Codex는 다음을 수행할 수 있다고 가정한다.

허용:

- 파일 삭제/이동/격리
- apt/pip/cmake 등 필요한 패키지 설치
- Vivado/Vitis/PetaLinux 실행
- PetaLinux가 없으면 설치 경로 탐색, settings.sh source, 설치 안내/자동화
- bootgen 또는 Vitis packaging 사용
- SD 카드 mount/remount/write
- BOOT.BIN/image.ub/rootfs에 파일 복사
- USB serial terminal 접속
- 보드 Linux shell 명령 실행
- 데모 앱 build/run

단 하나의 예외:

SD 카드 전체 포맷, 파티션 삭제, dd 쓰기처럼 host disk를 날릴 수 있는 destructive 작업은 반드시 lsblk 결과를 보여주고 사용자가 SD device path를 1회 확인한 뒤 수행한다.

장시간 실행 규칙:

Vivado/PetaLinux/synthesis/implementation/bitstream/buildroot/rootfs build는 장시간 작업이다.

Codex는 1-2분마다 tail을 보며 컨텍스트를 태우지 않는다.

로그는 파일로 redirect하고, 최소 10분 단위 또는 프로세스 종료 시점에만 확인한다.

보고는 마지막 80줄과 ERROR/CRITICAL WARNING/FAIL grep 요약만 한다.

3. 선형 실행표

단계	이름	주 담당	입력	산출물
S00	낡은 파일 격리/삭제	사용자+Codex	현재 repo	deprecated 폴더
S01	DMA 하드웨어 재설계	팀원 B+Codex	stream core, fake_gemv	DMA top RTL, E
S01.5	GPTalk.xpr 단일 active project 통합	사용자+Codex	기존 GPTalk.xpr, S01 DMA 산출물	GPTalk 전용 Tcl,
S02	라우팅/timing 복구	팀원 B+Codex	GPTalk.xpr DMA design	bitstream, XSA
S03	부트 복구/PetaLinux/SD 구성	팀원 B+Codex	XSA/bit/known-good FSBL	기준 BOOT, SD b
S04	보드 UIO/register/DMA-register smoke	사용자+Codex	booted board	register/DMA lo
S04.5	DMA buffer provider 확보	사용자+Codex	S04 PASS 보드	carveout 또는 pr
S05	C gemv_hw_test	팀원 A+Codex	DMA HW info, golden	gemv_hw_test
S06	runtime 모든 GEMV backend화	팀원 A+Codex	GGUF/layout/HW driver	smollm2_chat
S07	HDMI console 최종 demo	사용자+Codex	SD/board/app	demo log/video
S08	freeze	전원	final artifacts	release folder

작업 위치 구분:

보드-only:

S04, S04.5A, S04.5C, S05, S06/S07 실행

SD-only:

S03 boot 파일 배치

S04.5B 영구 bootargs 반영

PC/Vivado:

S01, S01.5, S02

팀원에게 넘기는 것은 긴 설명이 아니라 해당 단계 파일 묶음 + 해당 단계 Codex 프롬프트다.

S00. 낡은 AXI-Lite 데이터 경로 격리/삭제

육하원칙

왜:

현재 full wrapper가 AXI-Lite로 weight/input/output을 운반해서 최종 구조로 부적합하다.

누가:

사용자 또는 Codex.

언제:

지금 즉시. DMA 구조를 새로 만들기 전.

어디서:

/home/user22/Desktop/smollm2-zybo

무엇을:

낡은 AXI-Lite data-path 파일을 active build에서 제거하고 deprecated로 격리한다.

어떻게:

아래 삭제/격리 프롬프트를 Codex에 실행시킨다.

확인:

active scripts에서 INPUT_DATA/STREAM_DATA/RESULT_DATA 기반 최종 경로가 사라져야 한다.

Codex 프롬프트 - 삭제/격리

현재 프로젝트에서 낡은 AXI-Lite data-path 구조를 active build에서 제거하라.

중요:

완전 삭제 전에 deprecated/old_axi_lite_bringup/ 아래로 백업 이동한다.

로그와 증거는 남긴다.

계산 core인 vivado_ip/rtl/gemv_q8_0_stream_core.v는 절대 삭제하지 마라.

RTL 시뮬레이션에 필요한 testbench와 golden도 삭제하지 마라.

문제 구조:

- AXI-Lite INPUT_DATA 반복 write로 input vector 전송
- AXI-Lite STREAM_DATA 반복 write로 weight/scale stream 전송
- AXI-Lite RESULT_DATA 반복 read로 output vector 전송
- DMA 없는 create_zybo_gemv_hw.tcl / create_zybo_gemv_smoke_hw.tcl을 최종 구조로 사용하는 것
- smoke bitstream을 full GEMV bitstream으로 취급하는 것

보존할 파일:

- vivado_ip/rtl/gemv_q8_0_stream_core.v
- vivado_ip/tb/tb_gemv_q8_0_stream_core.sv
- vivado_ip/tb/tb_gemv_q8_0_stream_core.v, 있으면 보존
- scripts/run_gemv_sim.tcl
- golden/fake_gemv/*
- fpga_layout/q8_0_lane16/*
- runtime_c/*
- logs/gemv_sim_result.txt

active build에서 제거하거나 deprecated로 이동할 파일:

- vivado_ip/rtl/gemv_q8_0_axi_lite.v
- vivado_ip/rtl/gemv_q8_0_axi_lite_smoke.v
- scripts/create_zybo_gemv_hw.tcl
- scripts/create_zybo_gemv_smoke_hw.tcl

수정할 문서:

- docs/interface_contract.md 맨 위에 DEPRECATED 문구를 추가하라.
이유: AXI-Lite INPUT_DATA/STREAM_DATA/RESULT_DATA 경로는 최종 구조가 아니다.
- docs/rtl_next_step.md 맨 위에 DEPRECATED 문구를 추가하라.
이유: block_acc-only/CPU-scale 정책이 남아 있으면 현재 fixed-scale FPGA 정책과 충돌한다.

새로 만들 파일:

- docs/deprecated_axi_lite_data_path.md
- logs/s00_deprecate_axi_lite_path.txt

검증:

1. active scripts 디렉터리에 create_zybo_gemv_dma_hw.tcl이 아직 없으면 TODO로 표시한다.
2. grep -Rni "REG_STREAM_DATA\|REG_INPUT_DATA\|REG_RESULT_DATA\|STREAM_DATA\|INPUT_DATA\|RESULT_DATA" vivado_ip scrip
3. 위 키워드가 deprecated 폴더와 deprecated 문서에만 남아야 한다.
4. active build 문서에는 이 경로를 사용하지 말라는 문구가 있어야 한다.

금지:

- stream core 삭제 금지.
- fake_gemv golden 삭제 금지.
- 기존 로그 삭제 금지.
- 기능을 줄여 성공 처리 금지.

검증 프롬프트

S00 검증만 수행하라.

새 RTL을 작성하지 마라.

다음을 확인하라.

1. vivado_ip/rtl/gemv_q8_0_stream_core.v가 남아 있는가?
2. deprecated/old_axi_lite_bringup/ 아래에 기존 AXI-Lite wrapper와 smoke 파일이 이동되었는가?

3. active build에서 create_zybo_gemv_hw.tcl, create_zybo_gemv_smoke_hw.tcl이 최종 경로로 사용되지 않는가?
4. docs/interface_contract.md, docs/rtl_next_step.md 상단에 DEPRECATED 경고가 있는가?
5. STREAM_DATA/RESULT_DATA/INPUT_DATA 키워드가 active 최종 경로에 남아 있지 않은가?

결과를 logs/s00_verify_deprecated_path.txt에 작성하라.

S01. DMA 기반 하드웨어 구조 작성

육하원칙

- 왜:
AXI-Lite는 제어용이고 대용량 weight/scale/input/output 경로가 아니다.
- 누가:
팀원 B + Codex.
- 언제:
S00에서 낡은 경로를 active build에서 제거한 직후.
- 어디서:
Vivado 2024.2.2 사용 PC.
- 무엇을:
AXI DMA + AXIS FIFO + GEMV stream core + control AXI-Lite wrapper.
- 어떻게:
아래 강화 프롬프트를 Codex에 넣는다.
- 확인:
block design에 AXI DMA MM2S -> FIFO -> GEMV s_axis와 GEMV m_axis -> FIFO -> S2MM 또는 BRAM output이 있어야 한다.

Codex 프롬프트 - DMA 구조 및 라우팅 복구

현재 Zybo Z7-20 / Vivado 2024.2.2 프로젝트의 Q8_0 GEMV 하드웨어 구조를 복구한다.

현재 확인된 문제:

1. 현재 block design에는 AXI DMA가 없다.
2. scripts/create_zybo_gemv_hw.tcl과 scripts/create_zybo_gemv_smoke_hw.tcl은 PS7 + AXI interconnect + GEMV AXI-Lite 모
3. M_AXIS_MM2S -> GEMV stream input 경로가 없다.
4. S2MM 또는 output DMA 경로가 없다.
5. gemv_q8_0_axi_lite.v는 INPUT_DATA, STREAM_DATA, RESULT_DATA를 통해 대용량 데이터를 AXI-Lite로 전송한다.
6. 현재 smoke bitstream은 version/status register 접근용일 뿐 full GEMV datapath도 DMA path도 포함하지 않는다.
7. full GEMV AXI-Lite wrapper는 synthesis는 됐지만 route/timing 실패했다.
8. docs/rtl_next_step.md에는 예전 block_acc-only/CPU-scale 정책이 남아 있으므로 현재 정책과 충돌한다.

중요:

이 문제를 기능 축소로 해결하지 마라.

프로젝트 목표는 SmoLLM2 Transformer 내부 모든 Q8_0 GEMV를 FPGA backend로 offload하는 것이다.

따라서 q_proj 하나만 처리하거나, fake_gemv 전용 IP로 만들거나, mode=0/mode=1 중 하나를 제거하거나, scale 적용을 제거하거나,

보존할 핵심 파일:

- vivado_ip/rtl/gemv_q8_0_stream_core.v

이 파일은 s_axis_tdata/tvalid/tready/tlast 입력과 m_axis_tdata/tvalid/tready/tlast 출력을 가진 stream core이므로 계산 co

폐기/대체할 구조:

- AXI-Lite STREAM_DATA로 weight/scale을 밀어 넣는 구조
- AXI-Lite INPUT_DATA로 대량 input을 밀어 넣는 구조
- AXI-Lite RESULT_DATA로 output vector 전체를 읽는 구조
- smoke bitstream을 최종 GEMV bitstream으로 취급하는 구조

새 목표 구조:

Zynq PS

-> M_AXI_GPO

-> AXI-Lite control/status registers

-> AXI DMA control registers

-> 필요 시 AXI BRAM Controller control/data window

DDR

- > PS S_AXI_HP0
- > AXI DMA MM2S
- > AXIS Data FIFO
- > GEMV s_axis input

GEMV m_axis output

- > AXIS Data FIFO
- > AXI DMA S2MM
- > PS S_AXI_HP0
- > DDR output buffer

input_i16 vector 경로:

1차 권장:

AXI BRAM Controller + input BRAM

PS가 input vector를 BRAM address window에 burst/memcpy 방식으로 쓴다.

대체:

별도 AXIS input loader 또는 input packet header.

금지:

최종 구조에서 INPUT_DATA 단일 AXI-Lite register를 반복 write하는 방식.

AXI-Lite register map은 control/status 중심으로 축소한다:

- VERSION
- CONTROL
- STATUS
- ERROR_CODE
- MODE
- SCALE_SHIFT
- IN_FEATURES
- OUT_FEATURES
- INPUT_BASE 또는 INPUT_BRAM_BASE/selector
- WEIGHT_STREAM_LENGTH
- RESULT_LENGTH
- START
- DONE
- DEBUG_ROW
- DEBUG_BLOCK
- DEBUG_LANE

AXI-Lite에서 제거하거나 deprecated 처리할 것:

- INPUT_DATA
- STREAM_DATA
- STREAM_LAST
- RESULT_DATA
- RESULT_ROW
- RESULT_BLOCK
- RESULT_LANE
- RESULT_LAST

단, 디버깅용으로 남기더라도 최종 데모 경로에서는 사용 금지라고 문서화한다.

Vivado block design 요구:

1. processing_system7_0 생성.
2. PCW_USE_M_AXI_GP0 활성화.
3. PCW_USE_S_AXI_HP0 활성화.
4. FCLK_CLK0는 우선 50 MHz로 설정한다.
5. proc_sys_reset을 사용한다. FCLK_RESET0_N을 IP reset에 직접 연결하지 않는다.
6. AXI DMA IP를 추가한다.
7. AXI DMA는 simple mode 우선, scatter-gather 비활성화 가능.
8. AXI DMA S_AXI_LITE는 PS M_AXI_GP0에 연결한다.
9. AXI DMA M_AXI_MM2S와 M_AXI_S2MM은 PS S_AXI_HP0 DDR 경로에 연결한다.
10. AXI DMA M_AXIS_MM2S -> axis_data_fifo -> GEMV s_axis로 연결한다.

11. GEMV m_axis -> axis_data_fifo -> AXI DMA S_AXIS_S2MM으로 연결한다.
12. GEMV control/status AXI-Lite wrapper는 PS M_AXI_GP0에 연결한다.
13. input vector BRAM을 쓴다면 AXI BRAM Controller를 PS M_AXI_GP0에 연결하고 GEMV core의 input read port와 dual-port BRAM을 연결한다.
14. address map을 logs/hw_dma_address_map.txt에 저장한다.

Clock/timing 정책:

- 첫 route 성공 목표는 PL 50 MHz다.
- CPU/PS clock은 낮추지 않는다.
- GEMV core, AXI DMA, AXIS FIFO, AXI interconnect가 같은 FCLK_CLK0 또는 명확한 clock converter를 사용해야 한다.
- RTL의 X_INTERFACE_PARAMETER에 FREQ_HZ 100000000이 박혀 있으면 50 MHz 설정과 맞게 수정하거나 제거한다.
- 50 MHz 성공 후 75 MHz, 100 MHz 순서로 시도한다.

RTL timing 복구:

- gemv_q8_0_stream_core.v 기능은 유지한다.
- ST_BLOCK_DONE에서 16 lane의 block_acc * scale_q * round_shift * row_acc add를 한 사이클에 몰아넣지 말고 pipeline stage로 나누고 stage를 늘린다.
- 권장 stage:
 - Stage A: block_acc latch
 - Stage B: scale multiply
 - Stage C: rounding shift
 - Stage D: row_acc accumulation
- 16-lane MAC 기능은 유지한다.
- 필요하면 scale stage만 lane-wise 또는 4-lane group time-mux 할 수 있다. 단 모든 GEMV는 여전히 FPGA에서 처리되어야 하며 (FPGA에서 처리되어야 하는 것은 아니지만, 여기서는 그렇게 가정한다).
- 큰 buffer는 FF/LUT array가 아니라 BRAM/XPM memory로 강제한다.
- output vector 전체를 register array로 노출하지 않는다.
- 불필요한 keep/dont_touch/mark_debug를 제거한다.

반드시 만들 새 파일:

- vivado_ip/rtl/gemv_q8_0_dma_top.v
- vivado_ip/rtl/gemv_q8_0_ctrl_axi_lite.v
- scripts/create_zybo_gemv_dma_hw.tcl
- scripts/build_zybo_gemv_dma_bitstream.tcl
- scripts/report_failed_impl.tcl
- docs/hw_dma_architecture.md
- docs/hw_route_recovery.md
- docs/interface_contract_dma.md

수정 또는 deprecated 표시할 파일:

- docs/interface_contract.md
 - 기존 AXI-Lite INPUT_DATA/STREAM_DATA/RESULT_DATA 경로는 deprecated라고 표시한다.
- docs/rtl_next_step.md
 - 현재 정책과 충돌하므로 deprecated 문구를 맨 위에 추가한다.

report_failed_impl.tcl 요구:

실패한 implementation이 있으면 다음 report를 생성한다.

- reports/full_gemv_util_hier.rpt
- reports/full_gemv_timing_summary.rpt
- reports/full_gemv_route_status.rpt
- reports/full_gemv_congestion.rpt
- reports/full_gemv_qor_suggestions.rpt

장시간 Vivado 실행 규칙:

- Vivado/PetaLinux 명령은 로그를 파일로 redirect한다.
- 실행 중 진행상황을 1-2분마다 반복 확인하지 마라.
- 긴 실행을 시작하면 최소 10분 단위 또는 프로세스 종료 시점에만 확인한다.
- 중간 polling을 하더라도 전체 로그를 붙여넣지 말고 마지막 80줄만 본다.
- 최종 보고에는 전체 로그를 붙여넣지 말고, grep으로 ERROR/CRITICAL WARNING/FAIL만 요약한다.
- full log는 logs/*.txt 파일로 남긴다.
- 성공/실패 판단에 필요한 요약만 docs/hw_dma_bringup_result.md에 쓴다.

장시간 실행용 명령 예:

```
nohup /opt/Xilinx/Vivado/2024.2/bin/vivado -mode batch -source scripts/build_zybo_gemv_dma_bitstream.tcl > logs/vivado.log &
```

진행 확인은 다음만 사용:

```
ps -p $(cat logs/vivado_dma_build.pid)
tail -80 logs/vivado_dma_build_console.log
grep -Rni "ERROR\|CRITICAL WARNING\|Timing constraints are not met\|write_bitstream completed\|failed" logs hw 2>/dev
```

성공 기준:

1. full GEMV datapath가 포함된 bitstream이 생성된다.
2. AXI DMA MM2S -> GEMV s_axis 경로가 block design에 존재한다.
3. GEMV m_axis -> AXI DMA S2MM 또는 BRAM output path가 존재한다.
4. AXI-Lite는 control/status 용도로만 사용된다.
5. STREAM_DATA/RESULT_DATA 반복 MMIO 방식이 최종 경로에서 제거된다.
6. XSA가 export된다.
7. docs/hw_dma_bringup_result.md에 bitstream path, xsa path, GEMV base address, DMA base address, input/output buffer
8. 기능을 줄여 성공 처리하지 않는다.

실패 시 보고:

- WNS/TNS
- route congestion 위치
- hierarchical utilization 상위 10개 module
- 가장 긴 timing path 5개
- DMA BD 연결 상태
- AXI-Lite 데이터 경로가 남아 있는지 여부
- 다음 수정 제안

검증 프롬프트

S01 검증만 수행하라.

다음을 확인하고 logs/s01_dma_arch_verify.txt에 남겨라.

1. create_zybo_gemv_dma_hw.tcl이 존재하는가?
2. AXI DMA IP가 BD에 추가되는가?
3. PS S_AXI_HP0가 활성화되는가?
4. DMA M_AXIS_MM2S -> FIFO -> GEMV s_axis 연결이 있는가?
5. GEMV m_axis -> FIFO -> DMA S2MM 또는 명시적 output BRAM 경로가 있는가?
6. AXI-Lite는 control/status만 담당하는가?
7. INPUT_DATA/STREAM_DATA/RESULT_DATA가 final datapath에 남아 있지 않은가?
8. PL clock은 50 MHz부터 시작하는가?
9. stream core 기능과 mode=0/mode=1이 유지되는가?

실패하면 고치지 말고 먼저 무엇이 빠졌는지 목록으로 보고하라.

S01.5. GPTalk.xpr 단일 active project 통합

육하원칙

- 왜:
hw/vivado_project/GPTalk.xpr가 기존 GUI 기준 프로젝트인데, S01 과정에서 새 xpr가 생기면 프로젝트가 갈라진다.
사용자는 Vivado GUI에서 하나의 프로젝트만 열어야 한다.
- 누가:
사용자 + Codex.
- 언제:
S01 DMA 구조 생성 후, S02 bitstream 생성 전.
- 어디서:
PC/Vivado 프로젝트 root.
- 무엇을:
S01 DMA BD/RTL/scripts를 GPTalk.xpr 중심으로 통합한다.
- 어떻게:
새 xpr 생성 금지. GPTalk.xpr를 open_project해서 DMA BD를 생성/갱신한다.
- 확인:
hw 아래 active xpr는 GPTalk.xpr 하나로 고정되고, docs/00_ACTIVE_KR.md만 보면 다음 작업을 알 수 있어야 한다.

Codex 프롬프트 - GPTalk.xpr 통합

현재 프로젝트 정리 단계 S01.5를 수행하라.

상황:

- 기존 Vivado 프로젝트가 이미 존재한다.
 - hw/vivado_project/GPTalk.xpr
- S01 수행 중 새 Vivado 프로젝트가 생성되었을 수 있다.
- 앞으로 active Vivado project는 반드시 hw/vivado_project/GPTalk.xpr 하나만 사용한다.
- 새 .xpr 프로젝트를 추가로 만드는 것은 금지한다.

목표:

S01에서 만든 DMA 기반 GEMV block design, RTL, scripts를 기존 GPTalk.xpr 중심으로 통합한다.
사용자가 Vivado에서 열 프로젝트가 하나만 남도록 정리한다.

절대 금지:

- 또 다른 새 Vivado 프로젝트 생성 금지.
- hw/zybo_gemv_dma를 active project로 유지 금지.
- smoke register-only bitstream을 full GEMV 산출물로 취급 금지.
- AXI-Lite INPUT_DATA / STREAM_DATA / RESULT_DATA 기반 bulk data path 복구 금지.
- gemv_q8_0_stream_core.v 삭제 금지.
- mode=0 scaled output 제거 금지.
- mode=1 block_acc debug 제거 금지.
- lane 수 축소 금지.
- fake_gemv 전용 하드코딩 IP 생성 금지.

해야 할 일:

1. find hw -name "*.xpr" -print 결과를 확인한다.
2. active project를 hw/vivado_project/GPTalk.xpr로 고정한다.
3. S01 DMA 관련 RTL은 보존한다.
 - vivado_ip/rtl/gemv_q8_0_dma_top.v
 - vivado_ip/rtl/gemv_q8_0_ctrl_axi_lite.v
 - vivado_ip/rtl/gemv_q8_0_stream_core.v
4. GPTalk.xpr용 wrapper script를 만든다.
 - scripts/create_or_update_gptalk_dma_bd.tcl이 script는 hw/vivado_project/GPTalk.xpr를 open_project 해야 하며 새 project를 만들면 안 된다.
5. build script도 GPTalk.xpr 전용으로 만든다.
 - scripts/build_gptalk_dma_bitstream.tcl
6. 사용자용 현재 상태판 docs/00_ACTIVE_KR.md를 한국어로 갱신한다.
7. Vivado GUI용 docs/VIVADO_GUI_KR.md에는 사람이 GUI로 볼 절차만 적는다. Tcl 전문을 넣지 않는다.
8. S02 synthesis/implementation/bitstream은 아직 실행하지 않는다.

성공 기준:

- active Vivado project가 hw/vivado_project/GPTalk.xpr 하나로 고정된다.
- S01 DMA 구조가 GPTalk.xpr 안으로 들어간다.
- 새 xpr를 더 만들지 않는다.
- docs/00_ACTIVE_KR.md만 보면 다음에 뭘 실행할지 알 수 있다.

검증 프롬프트

S01.5 검증만 수행하라.

1. hw 아래 active .xpr가 GPTalk.xpr 하나인지 확인한다.
 2. deprecated project는 deprecated/vivado_projects 아래에만 있는지 확인한다.
 3. scripts/create_or_update_gptalk_dma_bd.tcl이 새 xpr를 만들지 않고 GPTalk.xpr를 open_project 하는지 확인한다.
 4. scripts/build_gptalk_dma_bitstream.tcl이 GPTalk.xpr에서 build하도록 되어 있는지 확인한다.
 5. docs/00_ACTIVE_KR.md가 한국어 상태판이고 장문 로그/프롬프트를 담고 있지 않은지 확인한다.
 6. docs/VIVADO_GUI_KR.md가 GUI 중심인지 확인한다.
- 결과를 logs/s01_5_gptalk_unification_verify.txt에 작성하라.

S02. bitstream/XSA 생성과 routing/timing

육하원칙

왜: 실제 보드에 올릴 full GEMV DMA bitstream이 필요하다.
누가: 팀원 B + Codex.
언제: S01에서 DMA block design이 작성된 뒤.
어디서: Vivado 2024.2.2 PC.
무엇을: full GEMV datapath 포함 bitstream과 XSA.
어떻게: 장시간 실행 규칙으로 batch build를 돌린다.
확인: write_bitstream 성공, XSA export, timing report 저장.

Codex 프롬프트

S02 bitstream/XSA 생성을 수행하라.

전제:
- S01에서 create_zybo_gemv_dma_hw.tcl과 build_zybo_gemv_dma_bitstream.tcl이 생성되어 있다.
- 목표는 full GEMV DMA datapath 포함 bitstream이다.
- smoke-only bitstream은 성공으로 치지 않는다.

작업:
1. logs, reports, hw/zybo_gemv_dma 폴더를 준비한다.
2. Vivado batch build를 nohup으로 실행한다.
3. 진행상황을 너무 자주 확인하지 않는다.
4. 완료 후 bitstream과 XSA 경로를 찾는다.
5. timing/utilization/congestion report를 reports/에 저장한다.
6. docs/hw_dma_bringup_result.md를 작성한다.

실행 예:
mkdir -p logs reports
nohup /opt/Xilinx/Vivado/2024.2/bin/vivado \
-mode batch \
-source scripts/build_zybo_gemv_dma_bitstream.tcl \
> logs/vivado_dma_build_console.log 2>&1 &
echo \$! > logs/vivado_dma_build.pid

완료 후 요약:
tail -80 logs/vivado_dma_build_console.log
grep -Rni "ERROR\|CRITICAL WARNING\|FAIL\|Timing constraints are not met\|write_bitstream\|failed" logs hw reports 2>

성공 기준:
- full GEMV datapath 포함 .bit 존재
- .xsa 존재
- AXI DMA base address와 GEMV ctrl base address 기록
- timing 실패 여부 명시
- smoke-only가 아님을 문서에 명시

검증 프롬프트

S02 산출물 검증만 수행하라.
새 기능 추가하지 마라.

확인:
1. 생성된 bitstream이 smoke-only가 아니라 DMA + full GEMV datapath를 포함하는가?
2. XSA가 export되었는가?

3. address map에 GEMV control, AXI DMA, input BRAM이 있으면 BRAM controller 주소가 있는가?
4. timing WNS/TNS가 기록되었는가?
5. write_bitstream 성공 로그가 있는가?
6. 실패했다면 report_failed_impl.tcl 결과가 생성되었는가?

결과를 logs/s02_bitstream_xsa_verify.txt에 작성하라.

S03. PetaLinux/SD/serial 자동화

육하원칙

왜:

bitstream/XSA를 보드 Linux에 반영해야 C 프로그램이 FPGA를 호출할 수 있다.

누가:

팀원 B + Codex. 사용자는 SD 삽입, 보드 전원, 필요 시 destructive SD 작업 승인만 한다.

언제:

S02 XSA가 나온 직후.

어디서:

PetaLinux/Vitis/bootgen 가능한 PC와 SD 카드가 연결된 PC.

무엇을:

B00T.BIN, image.ub, rootfs 파일, demo app 복사.

어떻게:

Codex가 설치/경로/SD read-only 문제를 해결하며 진행한다.

확인:

SD bootfs/rootfs에 새 파일이 들어가고 보드가 부팅된다.

Codex 프롬프트

S03 PetaLinux/SD/serial 자동화를 수행하라.

목표:

S02에서 나온 XSA/bitstream을 반영해 Zybo Z7-20이 새 GEMV DMA hardware로 부팅되게 만든다.

사용자가 할 일은 SD 카드 삽입, 보드에 SD 장착, 전원 인가뿐이다.

Codex는 가능한 범위에서 설치, mount, copy, serial terminal 조작까지 수행한다.

권한:

- 필요한 패키지 설치 허용.
- PetaLinux/Vitis/Vivado settings.sh 탐색 및 source 허용.
- PetaLinux가 없으면 설치 가능 여부를 확인하고 설치 경로를 제안/수행한다.
- bootgen으로 FSBL + bitstream + U-Boot B00T.BIN을 만드는 fallback 허용.
- SD card remount rw 허용.
- 파일 복사 허용.
- serial terminal 접속 허용.

안전:

- SD 전체 포맷, 파티션 삭제, dd write는 lsblk 결과를 보여주고 사용자가 SD device path를 1회 확인한 뒤 수행한다.
- host disk로 의심되는 장치에는 절대 쓰지 않는다.

작업 순서:

1. source 가능한 Vivado/Vitis/PetaLinux settings.sh를 찾는다.
2. petalinux-* 명령 존재 여부를 확인한다.
3. PetaLinux 가능하면 XSA를 반영해 project/config/build/package를 진행한다.
4. PetaLinux가 불가능하면 Vitis/bootgen fallback으로 B00T.BIN 생성 가능 여부를 확인한다.
5. SD mount 상태를 확인한다.
6. read-only면 원인을 확인하고 remount rw 또는 fsck/권한 문제를 해결한다.
7. bootfs에 B00T.BIN, image.ub 등 필요한 파일을 복사한다.
8. rootfs 또는 bootfs에 다음 파일을 복사한다.
 - smollm2_chat
 - gemv_hw_test
 - SmoLLM2-135M-Instruct-Q8_0.gguf 또는 경로 안내
 - fpga_layout/q8_0_lane16/*

- golden/fake_gemv/*
- 9. 사용자가 보드를 부팅하면 USB serial terminal로 접속한다.
- 10. uname, dmesg, ls /dev, devmem 가능 여부를 확인한다.
- 11. logs/s03_board_boot_log.txt에 부팅/serial 로그를 저장한다.

산출물:

- docs/sd_boot_packaging_result.md
- logs/s03_petalinux_or_bootgen_log.txt
- logs/s03_sd_copy_log.txt
- logs/s03_board_boot_log.txt

성공 기준:

- 보드 Linux가 새 bitstream/boot files로 부팅한다.
- serial shell 접근 가능하다.
- GEMV/DMA base address 확인 준비가 되어 있다.

검증 프롬프트

S03 검증만 수행하라.

확인:

1. 어떤 방식으로 boot image를 만들었는가? PetaLinux인가 bootgen fallback인가?
2. SD bootfs/rootfs에 무엇을 복사했는가?
3. read-only 문제는 해결되었는가?
4. serial terminal로 보드 shell에 접근했는가?
5. devmem 또는 대체 MMIO 접근 도구가 있는가?
6. dmesg에 DMA/UIO 관련 오류가 있는가?

결과를 docs/s03_boot_verify.md에 작성하라.

S03.5. 부트 복구 baseline 판정과 FSBL 계보 고정

목적

S03에서 boot가 꼬였을 때 PetaLinux full rebuild로 바로 가지 않고, FSBL/U-Boot/bitstream/SD/serial 계층을 분리한다. v5 기준으로 이미 다음 baseline이 성공했다.

test_c_s03_fsbl_active_bit_s03_uboot:

```
FSBL: artifacts/s03_bootgen/zynq_fsbl.elf
bitstream: hw/vivado_project/export/GPTalk_dma.bit
U-Boot: artifacts/s03_bootgen/u-boot.elf
결과: DONE LED on, Linux root prompt, UIO 노드 노출
```

Codex 프롬프트 - boot baseline 고정

S03.5 boot baseline을 고정하라.

현재 기준 BOOT:

- artifacts/boot_tests/test_c_s03_fsbl_active_bit_s03_uboot/BOOT.BIN

기준 구성:

- FSBL: artifacts/s03_bootgen/zynq_fsbl.elf
- bitstream: hw/vivado_project/export/GPTalk_dma.bit
- U-Boot: artifacts/s03_bootgen/u-boot.elf

판정:

- DONE LED on
- Linux root prompt 도달
- /sys/class/uio에 axi_dma, input_bram, gemv_ctrl, hdmi_vdma, hdmi_vtc, hdmi_dynclock 존재
- custom/recovery FSBL은 폐기 대상
- PetaLinux full rebuild는 S03 boot recovery에는 필요 없음

해야 할 일:

1. artifacts/boot_tests/test_c_s03_fsbl_active_bit_s03_uboot/MANIFEST.txt를 보존한다.
2. 기준 BOOT.BIN의 sha256을 docs/00_ACTIVE_KR.md에 기록한다.
3. custom/recovery FSBL과 관련 BOOT은 deprecated로 표시한다.
4. 이후 S04/S04.5/S05는 이 기준 BOOT로 부팅한 보드에서 진행한다.

금지:

- custom/recovery FSBL 재사용 금지.
- PetaLinux full rebuild로 되돌아가기 금지.
- Digilent demo boot/image와 custom 산출물 섞기 금지.

S04. 보드 UIO/register/BRAM/DMA-register smoke

목적

S03 기준 BOOT로 부팅한 보드에서 user-space가 실제로 GEMV 관련 MMIO, input BRAM, AXI DMA register에 접근할 수 있는지 확인한다. 이 단계는 **실제 DMA transfer가 아니라 control plane 검증**이다.

Codex 프롬프트

S04 보드 UIO/register/DMA-register smoke를 수행하라.

전제:

- S03 boot recovery baseline은 PASS다.
- 기준 BOOT는 artifacts/boot_tests/test_c_s03_fsbl_active_bit_s03_uboot/BOOT.BIN 이다.
- DONE LED on, Linux root prompt, /sys/class/uio 노드 확인 완료.
- custom/recovery FSBL은 폐기 대상이다.
- PetaLinux full rebuild는 하지 않는다.

목표:

S05 gemv_hw_test로 넘어가기 전에 user-space에서 GEMV 관련 MMIO/BRAM/DMA register 접근이 실제로 가능한지 검증한다.

해야 할 일:

1. /sys/class/uio/uio*/name, map0/addr, map0/size를 수집한다.
2. axi_dma, input_bram, gemv_ctrl를 uio 번호가 아니라 name으로 찾는다.
3. Vivado address map과 UIO addr/size를 비교한다.
4. gemv_ctrl UIO를 mmap해서 VERSION/STATUS/ERROR register를 읽는다.
5. input_bram UIO에 known pattern을 write/readback한다.
6. axi_dma UIO에서 MM2S/S2MM control/status register를 읽고 reset 후 idle/error 상태를 확인한다.
7. DMA buffer provider 존재 여부를 확인한다.
 - /dev/udmabuf*
 - /dev/dma_proxy*
 - /proc/device-tree/reserved-memory
 - /sys/class/dma_heap
8. 결과를 logs/s04_board_uio_register_dma_smoke.txt와 docs/s04_smoke_verify.md에 기록한다.

성공 기준:

- gemv_ctrl register read PASS
- input_bram write/readback PASS
- axi_dma register reset/status PASS
- bus error/kernel oops 없음

S05 진입 조건:

- 위 smoke PASS에 더해 DMA buffer provider 또는 carveout이 확정되어야 한다.

금지:

- UIO 번호 하드코딩 금지.
- register read만 하고 GEMV 성공이라고 쓰기 금지.
- DMA buffer/cache 미확정 상태에서 fake_gemv 실행 금지.
- SmoLLM2 runtime 실행 금지.
- PetaLinux full rebuild 금지.

v5 현재 S04 결과

PASS:

```
axi_dma:      uio0 0x40400000 size 0x10000
input_bram:   uio1 0x42000000 size 0x10000
gemv_ctrl:    uio5 0x43ca0000 size 0x1000
Vivado address map 비교 PASS
UIO lookup name 기반 PASS
GEMV VERSION=0x000a0001 STATUS=0 ERROR=0
input_bram write/readback PASS
AXI DMA reset/status register PASS
bus error/kernel oops 없음
```

BLOCKED:

usable user-space DMA buffer provider 없음

검증 프롬프트

S04 검증만 수행하라.

logs/s04_board_uio_register_dma_smoke.txt와 docs/s04_smoke_verify.md를 확인하고 다음을 판정하라.

- UIO map/address: PASS/FAIL
- gemv_ctrl register: PASS/FAIL
- input_bram read/write: PASS/FAIL
- axi_dma register reset/status: PASS/FAIL
- DMA buffer provider: 있음/없음
- S05로 바로 갈 수 있는가?

DMA buffer provider가 없으면 S04는 PASS지만 S05는 BLOCKED로 판정한다.

결과를 docs/s04_smoke_verify.md에 작성하라.

S04.5. DMA buffer provider 확보

목적

AXI DMA는 physical address를 요구한다. 현재 Linux에는 /dev/udmabuf, dma_proxy, reserved-memory, dma_heap 같은 user-space DMA buffer provider가 없다. 따라서 S05 fake_gemv 전에 DMA-safe physical buffer 경로를 만들어야 한다.

작업 분리 원칙

S04.5A:

보드-only
SD를 PC에 뽑지 않는다.
U-Boot에서 임시 bootargs mem=... 를 넣고 1회성 부팅 테스트.

S04.5C:

보드-only
mem= 부팅 후 /proc/iomem 확인, /dev/mem 0_SYNC mmap, carveout write/readback.

S04.5B:

SD-only
S04.5A/C가 성공한 뒤에만 수행.
검증된 mem= bootargs를 SD에 영구 반영한다.
BOOT.BIN은 바꾸지 않는다.

S04.5D:

fallback
carveout 방식이 실패하면 udmabuf/dma-proxy/reserved-memory 정식 provider를 검토한다.

추천 순서:

1. S04.5A 보드-only 임시 mem= 테스트
2. S04.5C 보드-only carveout /dev/mem smoke
3. 성공하면 S04.5B SD-only 영구 bootargs 반영
4. 그 다음 S05 fake_gemv

S04.5A Codex 프롬프트 - 보드-only 임시 mem= 테스트

S04.5A board-only 임시 mem= carveout 부팅 테스트를 수행하라.

현재 상태:

- S03 boot recovery baseline PASS
- S04 UI0/register/BRAM/DMA-register smoke PASS
- S05 fake_gemv는 DMA buffer provider 부재로 BLOCKED

목표:

SD 파일을 수정하지 않고, U-Boot에서 1회성 bootargs mem=... 를 넣어 Linux 메모리 제한 부팅이 가능한지 확인한다. 이 테스트는 DMA carveout 후보를 검증하기 위한 사전 단계다.

전제:

- 사용자는 보드에 현재 S03 baseline SD를 꽂고 전원을 넣는다.
- Codex는 serial terminal로 U-Boot autoboot를 중단할 수 있다고 가정한다.
- HDMI는 성공 기준이 아니다. serial console root prompt가 기준이다.

작업 순서:

1. serial terminal을 연다.
2. U-Boot autoboot를 중단한다.
3. 기존 bootargs와 bootcmd를 저장한다.
 - printenv bootargs
 - printenv bootcmd
4. DDR 크기와 기존 kernel load 관련 정보를 수집한다.
 - bdfinfo
 - printenv
5. mem= 후보를 계산한다.
 - DDR이 1GB로 확인되면 우선 mem=960M 후보를 사용한다.
 - 이 경우 carveout 후보는 대략 0x3C000000~0x3FFFFFFF, 64MB이다.
 - 실제 DDR size와 load address를 보고 겹치지 않는지 확인한다.
6. 기존 bootargs를 보존하면서 mem=... 만 추가한다.
7. saveenv는 하지 않는다.
8. boot를 실행한다.
9. Linux root prompt가 뜨면 다음을 수집한다.
 - cat /proc/cmdline
 - cat /proc/meminfo
 - cat /proc/iomem
 - dmesg | grep -Ei "Memory|memblock|Reserved|CMA|DMA|uio"
10. 결과를 logs/s04_5a_temp_mem_boot.txt에 저장한다.

성공 기준:

- serial console에서 Linux root prompt 도달
- /proc/cmdline에 mem=... 반영
- /proc/iomem에서 Linux System RAM이 mem 제한값까지만 잡힘
- carveout 후보 영역이 System RAM에 포함되지 않음
- UI0 노드가 여전히 존재함

실패 시:

- root prompt 미도달이면 원래 bootargs로 재부팅 가능한지 확인한다.
- 이 단계에서 SD를 수정하지 마라.
- PetaLinux rebuild 하지 마라.

S04.5C Codex 프롬프트 - 보드-only /dev/mem carveout smoke

S04.5C board-only /dev/mem carveout smoke를 수행하라.

전제:

- S04.5A에서 임시 mem= 부팅이 PASS했다.
- carveout 후보 물리 주소가 Linux System RAM 밖임을 확인했다.

목표:

carveout physical address를 /dev/mem 0_RDWR | 0_SYNC로 mmap해서 write/readback이 가능한지 확인한다.

작업:

1. artifacts/boot_tests/s04_5_dma_carveout_smoke.c를 작성한다.
2. 인자:
 - --phys-base
 - --size
 - --pattern-count
3. /dev/mem을 0_RDWR | 0_SYNC로 연다.
4. carveout 영역을 mmap한다.
5. 여러 offset에 32-bit pattern을 write/readback한다.
6. 실패 시 physical address, offset, expected, actual, errno를 출력한다.
7. bus error/kernel oops 여부를 확인한다.
8. 결과를 logs/s04_5c_dma_carveout_smoke.txt에 저장한다.

성공 기준:

- /dev/mem mmap 성공
- pattern write/readback PASS
- bus error/kernel oops 없음
- S05에서 쓸 physical buffer layout을 계산할 수 있음

주의:

- 이 방식은 빠른 MVP용이다.
- cache coherency 문제가 생기면 udmabuf/dma-proxy로 전환한다.
- malloc virtual address를 DMA physical address처럼 쓰지 마라.

S04.5B Codex 프롬프트 - SD-only 영구 bootargs 반영

S04.5B SD-only 영구 bootargs 반영을 수행하라.

전제:

- S04.5A 임시 mem= 부팅 PASS
- S04.5C /dev/mem carveout smoke PASS
- 선택한 mem= 값과 carveout physical base/size가 확정됨

목표:

검증된 mem= bootargs를 SD bootfs에 영구 반영한다.

필요:

- SD 카드를 PC에 꽂는다.
- 보드는 필요 없다.

작업:

1. lsblk -f로 SD 카드 장치를 확인한다.
2. SD bootfs/rootfs를 RW로 마운트한다.
3. 기존 boot config 파일을 backup/bootargs_YYYYMMDD_HHMMSS/ 아래에 백업한다.

후보:

 - boot.scr
 - uEnv.txt
 - extlinux/extlinux.conf
 - 기타 U-Boot env 파일
4. 현재 실제 부팅 방식이 무엇인지 확인한다.
5. 검증된 mem=... 을 기존 bootargs에 추가한다.
6. BOOT.BIN은 바꾸지 않는다.
7. image.ub/rootfs도 바꾸지 않는다.
8. SD를 sync 후 안전하게 unmount한다.

성공 기준:

- 다음 부팅에서 /proc/cmdline에 mem=... 이 자동 반영된다.

- Linux root prompt 도달.
- UIO 노드 유지.
- carveout 영역이 System RAM 밖이다.

금지:

- 검증 안 된 mem= 값을 SD에 박지 마라.
- BOOT.BIN 교체 금지.
- SD 전체 포맷, dd, 파티션 삭제 금지.

S04.5D fallback

carveout + /dev/mem 0_SYNC 방식이 실패하면 다음 중 하나를 선택한다.

1. udmabuf 추가
2. dma-proxy 추가
3. device tree reserved-memory + kernel driver 경로

이 단계는 PetaLinux 또는 DT 수정이 필요할 수 있으므로, S04.5A/C보다 나중에 한다.

S04.5 검증 프롬프트

S04.5 검증을 수행하라.

1. S04.5A 임시 mem= 부팅이 root prompt까지 갔는가?
2. /proc/cmdline에 mem= 값이 있는가?
3. /proc/iomem에서 carveout 영역이 System RAM 밖인가?
4. S04.5C /dev/mem mmap write/readback이 PASS했는가?
5. bus error/kernel oops가 없는가?
6. S04.5B를 수행했다면 SD 영구 bootargs 반영 후 재부팅에서도 동일한 상태인가?
7. S05에서 사용할 physical buffer layout이 확정됐는가?

결과를 docs/s04_5_dma_buffer_provider.md에 작성하라.

S05. C gemv_hw_test로 fake_gemv FPGA PASS

진입 조건

필수:

- S03 boot baseline PASS
- S04 UIO/register/BRAM/DMA-register smoke PASS
- S04.5 DMA buffer provider PASS
- S05에서 사용할 physical buffer layout 확정

금지:

- DMA buffer/cache 미확정 상태에서 fake_gemv 실행 금지

Codex 프롬프트

S05 gemv_hw_test를 작성/수정하고 보드에서 실행하라.

전제:

- S04.5 DMA buffer provider가 PASS해야 한다.
- reserved carveout 또는 동등한 DMA-safe physical buffer layout이 확정되어 있어야 한다.

목표:

보드 Linux C 프로그램이 AXI DMA를 통해 fake_gemv weight/scale/input을 FPGA GEMV IP에 보내고 output을 DDR로 받아 golden과

입력:

- golden/fake_gemv/*
- fpga_layout/q8_0_lane16/* 또는 fake_gemv packet
- docs/interface_contract_dma.md
- docs/hw_dma_bringup_result.md

작업:

1. linux_app 또는 runtime_c 아래에 gemv_hw_test C 프로그램을 둔다.
2. DMA MM2S buffer에 Q8_0 packet을 넣는다.
3. input vector는 설계된 input BRAM 또는 input buffer에 넣는다.
4. output buffer를 0xCD 패턴으로 초기화한다.
5. GEMV control register에 mode, in_features, out_features, scale_shift, lengths를 설정한다.
6. DMA MM2S/S2MM를 시작한다.
7. GEMV start를 건다.
8. timeout을 두고 done을 기다린다.
9. output cache invalidate가 필요하면 처리한다.
10. golden과 비교한다.
11. mode=0 scaled와 mode=1 block_acc를 모두 테스트한다.

출력 예:

```
[FPGA GEMV HW TEST]
case: fake_gemv
mode=0 scaled: PASS
mode=1 block_acc: PASS
```

성공 기준:

- FPGA가 실제 계산한다.
- AXI-Lite STREAM_DATA/RESULT_DATA 경로를 쓰지 않는다.
- DMA 또는 명시된 BRAM/output 경로를 쓴다.
- mode=0/mode=1 모두 PASS.

금지:

- fake_gemv 전용 RTL 하드코딩 금지.
- CPU reference만 실행하고 PASS 금지.
- AXI-Lite data register 반복 전송 금지.

검증 프롬프트

S05 검증을 수행하라.

확인:

1. gemv_hw_test가 AXI DMA 또는 명시된 BRAM 경로를 사용하는가?
2. AXI-Lite data register 반복 전송을 쓰지 않는가?
3. mode=0 scaled PASS가 있는가?
4. mode=1 block_acc PASS가 있는가?
5. timeout/error/fallback이 없는가?
6. CPU reference와 비교 로그가 있는가?

결과를 docs/s05_fake_gemv_hw_pass_verify.md에 작성하라.

S06. Runtime 모든 GEMV FPGA backend화

Codex 프롬프트

S06 SmolLM2 runtime의 모든 Q8_0 GEMV를 FPGA backend로 연결하라.

목표:

CPU-only가 아니라 FPGA 가속 chat runtime이다.

Transformer 내부 모든 Q8_0 2D matrix-vector GEMV 호출은 gemv_backend_run()을 통과하고, backend=fpga --require-fpga에서는

입력:

- SmolLM2-135M-Instruct-Q8_0.gguf
- fpga_layout/q8_0_lane16/*
- docs/interface_contract_dma.md
- gemv_hw_test에서 검증된 FPGA driver 코드
- runtime_c/*

작업:

1. smollm2_chat 또는 equivalent C runtime을 만든다.
2. GGUF 모델을 로드한다.
3. tokenizer/chat template은 가능한 기존 runtime 또는 llama.cpp 기반을 사용한다.
4. 모든 Q8_0 2D tensor GEMV 호출이 gemv_backend_run()을 지나가게 한다.
5. embedding lookup은 GEMV가 아니므로 CPU 허용.
6. RMSNorm/RoPE/softmax/sampling/KV cache는 CPU 허용.
7. lm_head도 GEMV이므로 FPGA로 보내는 것을 목표로 한다.
8. activation float32 -> int16 변환은 ACT_SHIFT=8부터 시작한다.
9. FPGA output_i32 -> float 복원은 output_float = output_i32 / (1 << ACT_SHIFT)로 시작한다.
10. --require-fpga에서는 CPU GEMV fallback이 1번이라도 발생하면 즉시 실패한다.
11. 매 응답마다 total_gemv_calls, fpga_gemv_calls, cpu_gemv_fallbacks를 출력한다.

실행:

```
./smollm2_chat \  
  --model /mnt/sd/SmolLM2-135M-Instruct-Q8_0.gguf \  
  --backend fpga \  
  --require-fpga \  
  --max-new-tokens 16
```

성공 기준:

- 대화가 출력된다.
- total_gemv_calls > 0
- fpga_gemv_calls == total_gemv_calls
- cpu_gemv_fallbacks == 0

금지:

- q_proj 하나만 offload하고 성공 금지.
- CPU-only 성공 금지.
- fallback 숨김 금지.
- 360M/Q4 금지.

검증 프롬프트

S06 검증을 수행하라.

확인:

1. 모든 Q8_0 GEMV call이 gemv_backend_run()을 통과하는가?
2. --require-fpga 모드가 있는가?
3. CPU GEMV fallback이 발생하면 즉시 실패하는가?
4. total_gemv_calls, fpga_gemv_calls, cpu_gemv_fallbacks가 출력되는가?
5. fpga_gemv_calls == total_gemv_calls인가?
6. fallback이 0인가?
7. 대화 출력이 실제로 생성되는가?

결과를 docs/s06_runtime_fpga_backend_verify.md에 작성하라.

S07. HDMI console 최종 demo

Codex 프롬프트

S07 최종 HDMI console demo를 준비하고 실행하라.

전제:

- 보드는 새 SD로 부팅된다.
- HDMI display와 USB keyboard가 연결되어 있다.
- serial terminal도 연결 가능하다.
- smollm2_chat과 model/layout/golden이 보드에 있다.

작업:

1. HDMI console에서 로그인 또는 자동 로그인 상태를 확인한다.

2. USB keyboard 입력을 확인한다.
3. serial terminal에서도 동시에 로그를 볼 수 있으면 유지한다.
4. smollm2_chat 실행 스크립트 run_demo.sh를 만든다.
5. max-new-tokens는 8 또는 16으로 고정한다.
6. demo prompt 2개를 준비한다.
7. 실행 결과를 logs/final_demo_run_1.txt, logs/final_demo_run_2.txt에 저장한다.
8. 재부팅 후 한 번 더 실행한다.

성공 기준:

- HDMI console에서 사용자가 문장을 입력한다.
- assistant 출력이 나온다.
- backend: fpga
- require_fpga: true
- fpga_gemv_calls == total_gemv_calls
- cpu_gemv_fallbacks == 0

검증 프롬프트

S07 검증을 수행하라.

확인:

1. HDMI console에서 실행되었는가?
2. USB keyboard로 입력했는가?
3. assistant 출력이 있는가?
4. FPGA backend counters가 출력되는가?
5. fallback이 0인가?
6. 재부팅 후 재현되었는가?

결과를 docs/s07_final_demo_verify.md에 작성하라.

S08. Freeze와 발표 리허설

새 기능 금지.

문서 수정은 실행 명령과 로그 위치만.

데모 SD를 복제하거나 최소한 BOOT.BIN/image.ub/rootfs/app/model을 백업한다.

체크리스트:

- [] bitstream/XSA path 기록
 - [] SD boot files 백업
 - [] smollm2_chat 실행 명령 고정
 - [] gemv_hw_test 실행 명령 고정
 - [] demo prompt 고정
 - [] fallback 0 로그 확보
 - [] serial terminal 로그 확보
 - [] HDMI console 사진/영상 확보
 - [] 재부팅 후 2회 재현
-

4. 팀원에게 넘길 것

팀원 B에게 넘길 것

파일:

```
전체 repo zip 또는 최소 hw_pack
vivado_ip/rtl/gemv_q8_0_stream_core.v
vivado_ip/tb/tb_gemv_q8_0_stream_core.sv
scripts/run_gemv_sim.tcl
golden/fake_gemv/*
fpga_layout/q8_0_lane16/*
```

프롬프트:

- S00 삭제/격리 프롬프트
- S01 DMA 구조 및 라우팅 복구 프롬프트
- S02 bitstream/XSA 프롬프트
- S03 PetaLinux/SD/serial 프롬프트

팀원 A에게 넘길 것

파일:

- 전체 repo zip 또는 최소 runtime_pack
- runtime_c/*
- fpga_layout/q8_0_lane16/*
- golden/fake_gemv/*
- Smollm2-135M-Instruct-Q8_0.gguf 경로
- S02/S03 결과로 나온 base address/register map/DMA map

프롬프트:

- S05 gemv_hw_test 프롬프트
- S06 runtime 모든 GEMV FPGA backend화 프롬프트
- S07 demo 프롬프트

5. 최종 판정표

상태	판정
XSim fake_gemv PASS만 있음	아직 보드 데모 아님
smoke bitstream version/status만 읽힘	PS-PL 주소 접근만 확인
AXI-Lite STREAM_DATA로 weight 전송	최종 구조 부적합
DMA MM2S/S2MM 포함 bitstream 생성	하드웨어 구조 통과 후보
gemv_hw_test fake_gemv PASS	FPGA GEMV 보드 실행 성공
smollm2_chat backend=fpga fallback 0	최종 데모 성공

6. 절대 금지 문구

Codex가 아래처럼 말하면 다시 시킨다.

- "일단 AXI-Lite로 weight를 써서 동작 확인"
- "DMA는 나중에"
- "q_proj 하나만 offload"
- "fallback이 있지만 데모는 성공"
- "smoke bitstream으로 보드 bring-up 성공"
- "scale은 CPU에서 처리"
- "mode=1 debug는 제거"

이번 프로젝트는 모든 Q8_0 GEMV를 FPGA backend로 보내는 DMA 기반 가속 데모다.

7. v5 반영 이력

v5는 v4를 축약한 찌꺼기 문서가 아니라, v4 전체 흐름에 다음 실전 변경을 병합한 전체판이다.

추가:

- S01.5 GPTalk.xpr 단일 active project 통합
- S03.5 boot baseline 고정
- S04 현재 PASS 결과 반영
- S04.5 DMA buffer provider 분리
- 보드-only / SD-only 작업 분리
- 문서 원칙, 사람용/agent용 분리

유지:

- S00 밝은 AXI-Lite 경로 격리
- S01 DMA 구조
- S02 bitstream/XSA
- S03 boot 구성
- S05 fake_gemv
- S06 runtime GEMV backend
- S07 HDMI demo
- S08 freeze